

Instructions:

- Non-programming calculator: Yes ☒ | No ☐
 - This paper consists of 3 sections and 6 practical questions.
 - **Section 1 (40 marks):** Question 1 (10 marks), Question 2 and 3 (15 marks).
 - **Section 2 (40 marks):** Question 1 (10 marks), Question 2 and 3 (15 marks).
 - **Section 3 (20 marks):** Two questions carrying 10 marks each.
-

DATABASE DESCRIPTION

You are managing the database for **TechNova Electronics Store**. The relational schema consists of the following tables:

- customer (customer_id, full_name, phone, email, registration_date)
- product (product_id, product_name, category, unit_price, stock_quantity)
- order_header (order_id, customer_id, order_date, status, total_amount)
- order_detail (detail_id, order_id, product_id, quantity, unit_price)
- payment (payment_id, order_id, payment_date, payment_method, amount)

Section 1/ (40 Marks) Data Querying - SQL DML

Question 1. (10 marks) Write an SQL statement to list all products (product_name, category, unit_price) that:

- a) Belong to category = 'Laptop' OR 'Tablet'
- b) AND have unit_price less than 25,000,000 VND
- c) Sort the result by unit_price ascending

Question 2. (15 marks) Write an SQL statement to generate a customer purchase report:

- a) The result must display: full_name, total_orders (number of orders), total_spent (sum of total_amount)
- b) Only include customers who have made at least 2 orders
- c) Sort the result by total_spent descending

Question 3. (15 marks) Write an SQL statement to identify “High-Value Customers”.

- a) A customer qualifies if:
 - They have purchased products in category = 'Laptop' at least once
 - OR Their total spending exceeds 100,000,000 VND
- b) The result must display: customer_id, full_name, email

Section 2/ (40 Marks) Database Programming - SQL Advanced

Requirement: Use Views, Procedures, and Triggers to handle business logic.

Question 1. (10 marks) - Create a View named v_order_invoice that shows: order_id, customer_name, order_date, total_product_amount (SUM of quantity × unit_price from order_detail), total_payment (SUM from payment table), balance (total_product_amount – total_payment).

Question 2. (15 marks) Write a Stored Procedure named sp_create_order.

- a) Parameters: p_customer_id, p_product_id, p_quantity
- b) Logic: Check if stock_quantity is sufficient. If not enough stock → Raise error: "Insufficient stock". If valid: Insert into order_header, Insert into order_detail, Update stock_quantity.

Question 3. (15 marks) Write a Trigger named trg_update_order_total.

- a) Event: AFTER INSERT on order_detail
- b) Action:
- Automatically recalculate total_amount in order_header
 - total_amount = SUM(quantity × unit_price)
- c) Condition: Only update if order status is NOT 'cancelled'

Section 3/ (20 Marks) Integration & Optimization

Requirement: Write DDL commands and provide short explanations.

Question 1. (10 marks) Currently, the system allows duplicate payments.

- a) Write a constraint or trigger logic to prevent a payment where the total payment amount for an order exceeds total_amount in order_header.
- b) Briefly explain how your solution enforces data integrity.

Question 2. (10 marks) Assume the order_header table contains 15 million records.

- a) Write an SQL command to create an Index to optimize this query:

```
SELECT *  
FROM order_header  
WHERE order_date BETWEEN '2025-01-01' AND '2025-12-31';
```

- b) Briefly explain why the following query may not use the index efficiently:

```
SELECT *  
FROM order_header  
WHERE YEAR(order_date) = 2025;  
----- End -----
```